

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

01 Nested Loop Structures

02 The “break” Statement

03 The “continue” Statement

04 The “pass” Statement

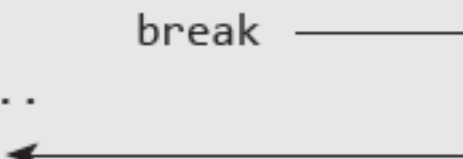
05 The “else” statement used with Loops

The “break” Statement

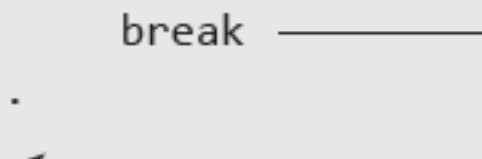
- The *break* statement is used to terminate the execution of the nearest enclosing loop in which it appears.
- When compiler encounters a break statement, the control passes to the statement that follows the loop in which the break statement appears.

The “break” Statement

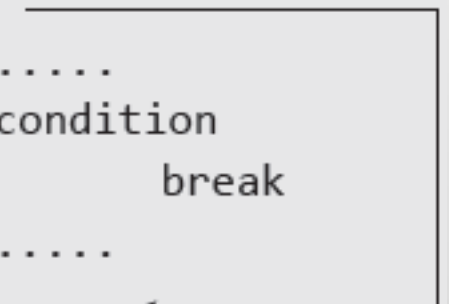
```
while...  
    .....  
    if condition:  
        .....  
        break  
    .....  
    .....  
Transfers control out of  
the while loop
```



```
for...  
    .....  
    if condition:  
        .....  
        break  
    .....  
.....  
Transfers control out of  
the for loop
```



```
for...  
    .....  
    for...  
        .....  
        if condition  
            .....  
            break  
        .....  
        .....  
Transfers control out of  
inner for loop
```



The “break” Statement Example

Program to Enter numbers until -1 is encountered

```
zeros=neg_num=pos_num=0
```

```
print("Enter -1 to exit..")
```

```
while(1):
```

```
    num=int(input("Enter any number: "))
```

```
    if(num==-1):
```

```
        break
```

```
    elif(num==0):
```

```
        zeros=zeros+1
```

```
    elif(num>0):
```

```
        pos_num=pos_num+1
```

```
    else:
```

```
        neg_num=neg_num+1
```

```
print("Count of Positive Numbers Entered: ",pos_num)
```

```
print("Count of Negative Numbers Entered: ",neg_num)
```

```
print("Count of Zeros Entered: ",zeros)
```

The “break” Statement Example

```
i = 1
while i <= 10:
    print(i, end=" ")
    if i==5:
        break
    i = i+1
print("\n Done")
```

OUTPUT

```
1 2 3 4 5
Done
```

The “break” Statement Example

```
for i in range(1,4+1):  
    for j in range(1,3+1):  
        if i==3:  
            break  
        print("*",end=' ')  
    print()
```

The “continue” Statement

- The “continue” statement can only appear in the body of a loop
- When the “continue” statement is encountered, the rest of the statements in that loop is skipped

The “continue” Statement Example

```
for i in range(1,11):  
    if(i==5):  
        continue  
    print(i, end=" ")  
print("\n Done")
```

OUTPUT

```
1 2 3 4 6 7 8 9 10  
Done
```

The “continue” Statement

```
while(...) ←  
    ...  
    If condition:  
        continue  
    ...  
...  
Transfers control to the condition  
expression of the while loop
```

```
for(...) ←  
    ...  
    if condition:  
        continue  
    ...  
...  
Transfers control to the condition  
expression of the for loop
```

```
for(...)  
    ...  
    for(...) ←  
        ...  
        if condition:  
            continue  
        ...  
    ...  
Transfers control to the condition  
expression of the inner for loop
```

The “continue” Statement

- Note that the code is meant to print numbers from 1 to 10
- But as soon as $i=5$, the “continue” statement is encountered
- So the rest of the statement inside the for loop is skipped

The “continue” Statement

- The “continue” statement is usually used to restart a statement sequence when an error occurs
- It stops the current iteration of the loop and continues with the next one

The “pass” Statement

- The “pass” statement is used when a statement is syntactically required but no command has to be executed
- It specifies null operation or simply No Operation (NOP)
- Nothing happens when the “pass” statement is executed

The “pass” Statement Example

Example prog to show "pass" Statement

```
for letter in "HELLO":  
    pass      #The statement is doing nothing  
    print("Pass : ", letter)  
print("Done")
```

OUTPUT

```
Pass :  H  
Pass :  E  
Pass :  L  
Pass :  L  
Pass :  O  
Done
```

The “pass” Statement

- **Difference between comment and pass statements:**
- In Python programming, pass is a null statement.
- The difference between a comment and pass statement is that while the interpreter ignores a comment entirely, pass is not ignored.
- Comment is not executed but pass statement is executed but nothing happens.

The “else” Statement with Loops

- In Python you can have an “else” statement associated with a loop
- The “else” statements associated with “for loop” is executed when the loop has completed iterating
- The “else” statements associated with “while loop” is executed when the condition becomes False

The “else” Statement with Loops

Example prog to show "else" Statement along with Loops

```
for letter in "HELLO":  
    print(letter, end=" ")  
else:  
    print("Done")
```

OUTPUT

H E L L O Done

```
i = 1  
while(i<0):  
    print(i)  
    i = i - 1  
else:  
    print(i, "is not negative so  
loop did not execute")
```

OUTPUT

1 is not negative so loop did not execute